

WEEK6-SKILL:

TASK1:

To Store Commands, Maintain History Buffer, Handle Capacity Limits, Retrieve Entries, Display History, Validate Consistency

```
UW PICO 5.09 File: history.c Modified
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_HISTORY 5
#define MAX_COMMAND 100

char history[MAX_HISTORY][MAX_COMMAND];
int history_count = 0;

void add_history(char command[])
{
    if (history_count < MAX_HISTORY)
    {
        strcpy(history[history_count], command);
        history_count++;
    }
    else
    {
        for (int i = 0; i < MAX_HISTORY - 1; i++)
        {
            strcpy(history[i], history[i + 1]);
        }
        strcpy(history[MAX_HISTORY - 1], command);
    }
}

void display_history()
{
    if (history_count == 0)
    {
        printf("History is empty\n");
        return;
    }
    for (int i = 0; i < history_count; i++)
    {
        printf("%d %s\n", i + 1, history[i]);
    }
}

void retrieve_history(int number)
{
    if (number < 1 || number > history_count)
    {
        printf("Invalid history number\n");
        return;
    }
    printf("%s\n", history[number - 1]);
}

int main()
{
    char command[MAX_COMMAND];
    while (1)
    {
        Get Help      WriteOut      Read File      Prev Pg
        Exit          Justify        Where is       Next Pg
        Cut Text      UnCut Text    Cur Pos       To Spell
    }
```

```
UW PICO 5.09 File: history.c Modified
if (history_count == 0)
{
    printf("History is empty\n");
    return;
}

for (int i = 0; i < history_count; i++)
{
    printf("%d %s\n", i + 1, history[i]);
}
}

void retrieve_history(int number)
{
    if (number < 1 || number > history_count)
    {
        printf("Invalid history number\n");
        return;
    }

    printf("%s\n", history[number - 1]);
}

int main()
{
    char command[MAX_COMMAND];

    while (1)
    {
        printf("myshell> ");

        fgets(command, sizeof(command), stdin);

        command[strcspn(command, "\n")] = '\0';

        if (strlen(command) == 0)
        {
            continue;
        }

        if (strcmp(command, "exit") == 0)
        {
            break;
        }

        if (strcmp(command, "history") == 0)
        {
            display_history();
            continue;
        }

        if (strncmp(command, "history ", 8) == 0)
        {
            int number;

            if (sscanf(command + 8, "%d", &number) == 1)
            {
                retrieve_history(number);
            }
        }
    }
}

Get Help WriteOut Read File Prev Pg Cut Text Cur Pos
Exit Justify Where is Next Pg UnCut Text To Spell
```

```
UW PICO 5.09 File: history.c Modified

fgets(command, sizeof(command), stdin);

command[strcspn(command, "\n")] = '\0';

if (strlen(command) == 0)
{
    continue;
}

if (strcmp(command, "exit") == 0)
{
    break;
}

if (strcmp(command, "history") == 0)
{
    display_history();
    continue;
}

if (strncmp(command, "history ", 8) == 0)
{
    int number;

    if (sscanf(command + 8, "%d", &number) == 1)
    {
        retrieve_history(number);
    }
    else
    {
        printf("Invalid history command\n");
    }
}

continue;

add_history(command);

printf("Command stored: %s\n", command);
}

return 0;
}

Get Help WriteOut Read File Prev Pg Cut Text Cur Pos
Exit Justify Where is Next Pg UnCut Text To Spell
```

```
vishalkarnati@VISHALS-MacBook-Air week5skill % cd ~/Desktop/os
vishalkarnati@VISHALS-MacBook-Air os % mkdir week6skill
vishalkarnati@VISHALS-MacBook-Air os % cd week6skill
vishalkarnati@VISHALS-MacBook-Air week6skill % nano history.c
vishalkarnati@VISHALS-MacBook-Air week6skill % gcc history.c -o history
vishalkarnati@VISHALS-MacBook-Air week6skill % ./history
myshell> ls
Command stored: ls
myshell> pwd
Command stored: pwd
myshell> cd..
Command stored: cd..
myshell> hello
Command stored: hello
myshell> history
1  ls
2  pwd
3  cd..
4  hello
myshell> history 2
pwd
myshell> history 4
hello
myshell> exit
vishalkarnati@VISHALS-MacBook-Air week6skill %
```

TASK2:

To Create Pipeline Structures, Store Commands, Maintain Execution Order, Connect Processes, Validate Pipeline Layout, Test Configurations

```
UW PICO 5.09 File: pipeline.c Modified
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

#define MAX_COMMANDS 10
#define MAX_ARGS 20

int main()
{
    char input[500];
    char *commands[MAX_COMMANDS];
    int command_count = 0;

    printf("myshell> ");

    fgets(input, sizeof(input), stdin);
    input[strcspn(input, "\n")] = '\0';
    char *command = strtok(input, "|");

    while (command != NULL && command_count < MAX_COMMANDS)
    {
        commands[command_count] = command;
        command_count++;
        command = strtok(NULL, "|");
    }

    if (command_count == 0)
    {
        printf("Invalid pipeline\n");
        return 1;
    }

    for (int i = 0; i < command_count; i++)
    {
        while (*commands[i] == ' ')
        {
            commands[i]++;
        }
    }

    int pipes[MAX_COMMANDS - 1][2];
    pid_t pids[MAX_COMMANDS];

    for (int i = 0; i < command_count - 1; i++)
    {
        if (pipe(pipes[i]) == -1)
        {
            perror("pipe");
            return 1;
        }
    }

    for (int i = 0; i < command_count; i++)
    {
        if (command_count == 0)
        {
            printf("Invalid pipeline\n");
            return 1;
        }

        for (int i = 0; i < command_count; i++)
        {
            while (*commands[i] == ' ')
            {
                commands[i]++;
            }
        }

        int pipes[MAX_COMMANDS - 1][2];
        pid_t pids[MAX_COMMANDS];

        for (int i = 0; i < command_count - 1; i++)
        {
            if (pipe(pipes[i]) == -1)
            {
                perror("pipe");
                return 1;
            }
        }

        for (int i = 0; i < command_count; i++)
        {
            pids[i] = fork();

            if (pids[i] == -1)
            {
                perror("fork");
                return 1;
            }

            if (pids[i] == 0)
            {
                char *args[MAX_ARGS];
                int arg_count = 0;

                char *token = strtok(commands[i], " ");

                while (token != NULL && arg_count < MAX_ARGS - 1)
                {
                    args[arg_count] = token;
                    arg_count++;
                    token = strtok(NULL, " ");
                }

                args[arg_count] = NULL;

                if (i > 0)
                {
                    dup2(pipes[i - 1][0], STDIN_FILENO);
                }
            }
        }
    }
}
```

```
UW PICO 5.09 File: pipeline.c Modified

if (command_count == 0)
{
    printf("Invalid pipeline\n");
    return 1;
}

for (int i = 0; i < command_count; i++)
{
    while (*commands[i] == ' ')
    {
        commands[i]++;
    }
}

int pipes[MAX_COMMANDS - 1][2];
pid_t pids[MAX_COMMANDS];

for (int i = 0; i < command_count - 1; i++)
{
    if (pipe(pipes[i]) == -1)
    {
        perror("pipe");
        return 1;
    }
}

for (int i = 0; i < command_count; i++)
{
    pids[i] = fork();

    if (pids[i] == -1)
    {
        perror("fork");
        return 1;
    }

    if (pids[i] == 0)
    {
        char *args[MAX_ARGS];
        int arg_count = 0;

        char *token = strtok(commands[i], " ");

        while (token != NULL && arg_count < MAX_ARGS - 1)
        {
            args[arg_count] = token;
            arg_count++;
            token = strtok(NULL, " ");
        }

        args[arg_count] = NULL;

        if (i > 0)
        {
            dup2(pipes[i - 1][0], STDIN_FILENO);
        }
    }
}
```

```
UW PICO 5.09 File: pipeline.c Modified

if (pids[i] == -1)
{
    perror("fork");
    return 1;
}

if (pids[i] == 0)
{
    char *args[MAX_ARGS];
    int arg_count = 0;

    char *token = strtok(commands[i], " ");

    while (token != NULL && arg_count < MAX_ARGS - 1)
    {
        args[arg_count] = token;
        arg_count++;

        token = strtok(NULL, " ");
    }

    args[arg_count] = NULL;

    if (i > 0)
    {
        dup2(pipes[i - 1][0], STDIN_FILENO);
    }

    if (i < command_count - 1)
    {
        dup2(pipes[i][1], STDOUT_FILENO);
    }

    for (int j = 0; j < command_count - 1; j++)
    {
        close(pipes[j][0]);
        close(pipes[j][1]);
    }

    execvp(args[0], args);

    perror("exec");
    exit(1);
}

for (int i = 0; i < command_count - 1; i++)
{
    close(pipes[i][0]);
    close(pipes[i][1]);
}

for (int i = 0; i < command_count; i++)
{
    waitpid(pids[i], NULL, 0);
}

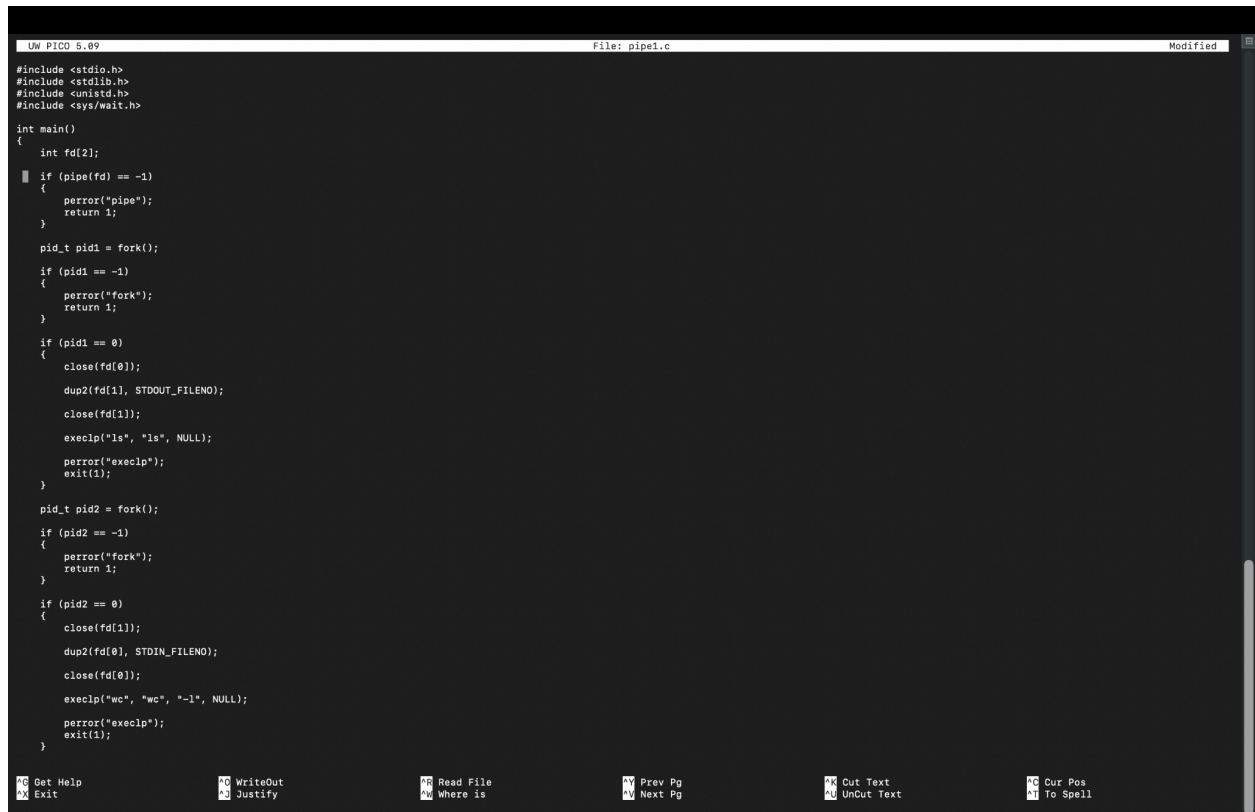
return 0;

⌘ Get Help ⌘ WriteOut ⌘ Read File ⌘ Prev Pg ⌘ Cut Text ⌘ Cur Pos
⌘ Exit ⌘ Justify ⌘ Where is ⌘ Next Pg ⌘ UnCut Text ⌘ To Spell
```

```
vishalkarnati@VISHALS-MacBook-Air week6skill % nano pipeline.c
vishalkarnati@VISHALS-MacBook-Air week6skill % gcc pipeline.c -o pipeline
vishalkarnati@VISHALS-MacBook-Air week6skill % ./pipeline
myshe11> ls | wc
4 38
vishalkarnati@VISHALS-MacBook-Air week6skill % ./pipeline
myshe11> ls | grep .c
history.c
pipeline.c
vishalkarnati@VISHALS-MacBook-Air week6skill % ./pipeline
myshe11> ls | grep .c | wc -l
2
vishalkarnati@VISHALS-MacBook-Air week6skill %
```

TASK3:

To Create Pipes, Redirect Streams, Execute Connected Commands, Close Descriptors, Synchronize Processes, Test Data Flow



```
UW PIC0 5.09 File: pipe1.c Modified
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    int fd[2];
    if (pipe(fd) == -1)
    {
        perror("pipe");
        return 1;
    }

    pid_t pid1 = fork();
    if (pid1 == -1)
    {
        perror("fork");
        return 1;
    }

    if (pid1 == 0)
    {
        close(fd[0]);
        dup2(fd[1], STDOUT_FILENO);
        close(fd[1]);
        execlp("ls", "ls", NULL);
        perror("execlp");
        exit(1);
    }

    pid_t pid2 = fork();
    if (pid2 == -1)
    {
        perror("fork");
        return 1;
    }

    if (pid2 == 0)
    {
        close(fd[1]);
        dup2(fd[0], STDIN_FILENO);
        close(fd[0]);
        execlp("wc", "wc", "-l", NULL);
        perror("execlp");
        exit(1);
    }

    wait(&pid1);
    wait(&pid2);
}
```

Get Help WriteOut Read File Prev Pg Cut Text Cur Pos
Exit Justify Where is Next Pg UnCut Text To Spell

```
UW PICO 5.09 File: pipe1.c Modified

    execlp("ls", "ls", NULL);
    perror("execlp");
    exit(1);
}

pid_t pid2 = fork();
if (pid2 == -1)
{
    perror("fork");
    return 1;
}

if (pid2 == 0)
{
    close(fd[1]);

    dup2(fd[0], STDIN_FILENO);
    close(fd[0]);

    execlp("wc", "wc", "-l", NULL);
    perror("execlp");
    exit(1);
}

close(fd[0]);
close(fd[1]);

waitpid(pid1, NULL, 0);
waitpid(pid2, NULL, 0);

printf("Pipeline completed\n");

return 0;
}

```

Get Help WriteOut Read File Prev Pg Cut Text Cur Pos
Exit Justify Where is Next Pg UnCut Text To Spell

```
vishalkarnati@VISHALS-MacBook-Air week6skill % nano pipe1.c
vishalkarnati@VISHALS-MacBook-Air week6skill % gcc pipe1.c -o pipe1
vishalkarnati@VISHALS-MacBook-Air week6skill % ./pipe1
8
Pipeline completed
vishalkarnati@VISHALS-MacBook-Air week6skill %
```

TASK4:

To Support Multiple Pipes, Manage Descriptor Arrays, Launch Multiple Processes, Coordinate Execution, Cleanup Resources, Test Long Pipelines

```
UW PICO 5.89 File: pipe2.c Modified
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <string.h>

int main()
{
    int pipe1[2];
    int pipe2[2];

    if (pipe(pipe1) == -1)
    {
        perror("pipe1");
        return 1;
    }

    if (pipe(pipe2) == -1)
    {
        perror("pipe2");
        return 1;
    }

    pid_t pid1 = fork();
    if (pid1 == 0)
    {
        close(pipe1[0]);
        dup2(pipe1[1], STDOUT_FILENO);
        close(pipe1[1]);
        close(pipe2[0]);
        close(pipe2[1]);
        execlp("ls", "ls", NULL);
        perror("ls");
        exit(1);
    }

    pid_t pid2 = fork();
    if (pid2 == 0)
    {
        close(pipe1[1]);
        dup2(pipe1[0], STDIN_FILENO);
        close(pipe1[0]);
        close(pipe2[0]);
        dup2(pipe2[1], STDOUT_FILENO);
        close(pipe2[1]);
        execlp("grep", "grep", ".c", NULL);
    }
}
```

Get Help WriteOut Read File Prev Pg Out Text Cur Pos
Exit Justify Where is Next Pg UnCut Text To Spell

```
UW PICO 5.09 File: pipe2.c Modified

    close(pipe1[1]);
    close(pipe2[0]);
    close(pipe2[1]);

    execlp("ls", "ls", NULL);

    perror("ls");
    exit(1);
}

pid_t pid2 = fork();
if (pid2 == 0)
{
    close(pipe1[1]);
    dup2(pipe1[0], STDIN_FILENO);

    close(pipe1[0]);
    close(pipe2[0]);

    dup2(pipe2[1], STDOUT_FILENO);
    close(pipe2[1]);

    execlp("grep", "grep", ".c", NULL);

    perror("grep");
    exit(1);
}

pid_t pid3 = fork();
if (pid3 == 0)
{
    close(pipe1[0]);
    close(pipe1[1]);
    close(pipe2[1]);

    dup2(pipe2[0], STDIN_FILENO);
    close(pipe2[0]);

    execlp("wc", "wc", "-l", NULL);

    perror("wc");
    exit(1);
}

close(pipe1[0]);
close(pipe1[1]);
close(pipe2[0]);
close(pipe2[1]);

Get Help WriteOut Read File Prev Pg Cut Text Cur Pos
Exit Justify Where is Next Pg UnCut Text To Spell
```

```
UW PICO 5.09 File: pipe2.c Modified

    perror("grep");
    exit(1);
}

pid_t pid3 = fork();
if (pid3 == 0)
{
    close(pipe1[0]);
    close(pipe1[1]);
    close(pipe2[1]);

    dup2(pipe2[0], STDIN_FILENO);
    close(pipe2[0]);

    execlp("wc", "wc", "-l", NULL);

    perror("wc");
    exit(1);
}

close(pipe1[0]);
close(pipe1[1]);

close(pipe2[0]);
close(pipe2[1]);

waitpid(pid1, NULL, 0);
waitpid(pid2, NULL, 0);
waitpid(pid3, NULL, 0);

printf("Long pipeline completed\n");

return 0;
}
```

```
vishalkarnati@VISHALS-MacBook-Air week6skill % nano pipe2.c
vishalkarnati@VISHALS-MacBook-Air week6skill % gcc pipe2.c -o pipe2
vishalkarnati@VISHALS-MacBook-Air week6skill % ./pipe2
4
Long pipeline completed
vishalkarnati@VISHALS-MacBook-Air week6skill %
```